

AI ASSISTED CODING

Lab Experiment: Prompt Engineering – Improving Prompts and Context Management.

NAME: G. Vijaya Simha Reddy

ROLLNO:2503A52671

BATCH:23

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a **zero-shot prompt** (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Zero-Shot Prompt:

```
def is_palindrome(num):  
    original = str(num)  
    reverse = original[::-1]  
  
    if original == reverse:  
        return "Palindrome"  
    else:  
        return "Not a Palindrome"  
  
n = int(input("Enter a number: "))  
print(is_palindrome(n))
```

Logical Errors / Edge Cases

- Negative numbers are not treated as palindromes.
- Non-integer input causes an error.
- Input validation can be improved.

Question 2: One-Shot Prompting (Factorial Calculation)

Write a **one-shot prompt** by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

Python Code:

```
def factorial(n):
    if n < 0:
        return "Factorial not defined"
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

num = int(input("Enter a number: "))
print(factorial(num))
```

Sample Output:

Input	Output
5	120

Comparison with Zero-Shot:

Feature	Zero-
---------	-------

	Shot	One-Shot	
Example Given			Yes
		No	
Clarity	Good	Better	
Correctness			More
	Basic	Accurate	
Readability			
	Normal	Improved	

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a **few-shot prompt** by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Task:

- Analyse how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Python Code:

```
def is_armstrong(num):
    digits = len(str(abs(num)))
    temp = abs(num)
    total = 0

    while temp > 0:
        digit = temp % 10
        total += digit ** digits
        temp //= 10
```

```

    if abs(num) == total:
        return "Armstrong Number"
    else:
        return "Not an Armstrong Number"

n = int(input("Enter a number: "))
print(is_armstrong(n))

```

Analysis

Multiple examples helped the AI generate a cleaner algorithm using digit counting and exponentiation, making the code more accurate.

Question 4: Context-Managed Prompting (Prime, Composite, Neither)

```

import math.

def classify_number(num):
    if num <= 1:
        return "Neither Prime nor Composite"

    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return "Composite"

    return "Prime"

try:
    n = int(input("Enter a number: "))
    print(classify_number(n))

```

```
except ValueError:
```

```
    print("Please enter a valid integer.")
```

Comparison

- Includes input validation.
- Uses square-root optimization.
- More efficient than checking every number.

Question 5: Zero-Shot Prompting (Perfect Number)

```
def is_perfect(num):
```

```
    if num <= 0:
```

```
        return "Not a Perfect Number"
```

```
    total = 0
```

```
    for i in range(1, num):
```

```
        if num % i == 0:
```

```
            total += i
```

```
    if total == num:
```

```
        return "Perfect Number"
```

```
    else:
```

```
        return "Not a Perfect Number"
```

```
n = int(input("Enter a number: "))
```

```
print(is_perfect(n))
```

Missing Conditions

- The loop checks up to num-1, which is inefficient.
- It can be optimized by checking divisors up to $\sqrt{n}$.

Question 6: Few-Shot Prompting (Even or Odd with Validation)

Python Code:

```
def even_odd():  
    try:  
        num = int(input("Enter a number: "))  
  
        if num % 2 == 0:  
            print("Even")  
        else:  
            print("Odd")  
  
    except ValueError:  
        print("Invalid input. Please enter an integer.")  
  
even_odd()
```

Analysis

The provided examples helped produce:

- Clearer output.
- Proper integer validation.
- Correct handling of zero and negative numbers.

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a **context-managed prompt** with clear instructions and constraints to generate an optimized Python program that classifies a number as **prime, composite, or neither**.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.